Who's Governing the GPU Lane? Policy as Code for AI Inference Traffic in Kubernetes

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER



Ciara Stacke
Tech Lead, NGINX Kubernetes Solutions
F5 / NGINX



Alessandro Fael Garcia
NGINX Community Evangelist
F5 / NGINX

So who’s governing the GPU lane?

The actual problem: RBAC decides who may create an object, not what's in it. In the AI era, we need more granular controls.

Teams share Services with ReferenceGrants. How can we stop them sharing models?

A grant opens a namespace to another one, for anything.

How can we keep teams on the models they're approved for?

A route can point at any pool it can reach.

How can we stop teams running models on top-end GPUs nobody pays for?

Anyone who can create a Deployment can ask for an H100.

How can we stop teams switching off the platform's failure mode?

Anyone who owns a pool can set it to keep serving without the endpoint picker.

Enter team-assistant and team-research

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER

models · shared
run by the platform team

llama-8b · small model · L4

llama-70b · large model · H100

8B, 70B: Billions of parameters. Bigger models answer better and need bigger GPUs.

L4, H100: Entry-level and top-end NVIDIA GPUs.

team-assistant · customer support
standard tier · regulated · llama-8b only

assistant route

Its goal: **Assistant Pro**, better answers for enterprise customers

Tier: What the platform lets a team use. Only premium teams get H100s.

Regulated: Assistant answers customers, so a request that can't be checked is refused.

team-research · data science
premium tier · may use every model

research route

research-ft · its own model · H100

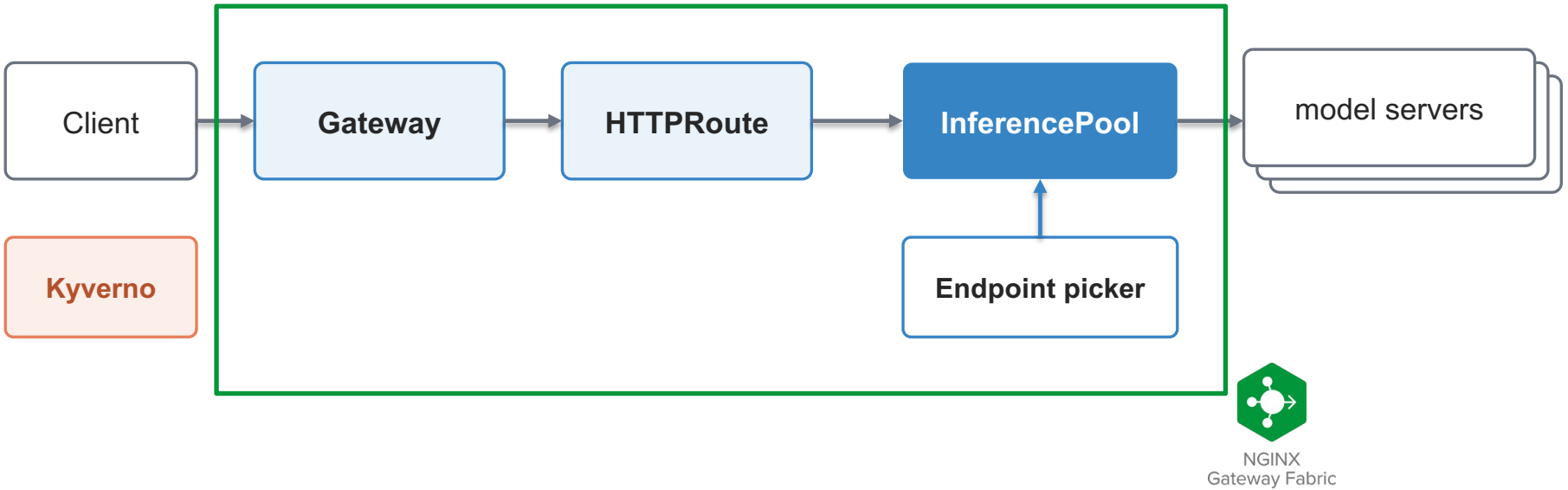
research-ft: a model team-research fine-tuned on internal data, including pricing that isn't public yet.

The demo: four scenarios

team-assistant tries four workarounds to get a better model

1	**Borrow a model** Scenario 1 · Tenant isolation	Uses team-research's fine-tuned model RBAC lets it create a ReferenceGrant
2	**Just point at the big one** Scenario 2 · Inference routing	Uses the platform's large model RBAC lets it create an HTTPRoute
3	**Shadow GPU** Scenario 3 · Cost attribution	Uses its own large model, on an H100 RBAC lets it create a Deployment
4	**Keep serving** Scenario 4 · Failover	Uses FailOpen, to keep serving if the picker is down RBAC lets it create an InferencePool, and edit it

The reference architecture

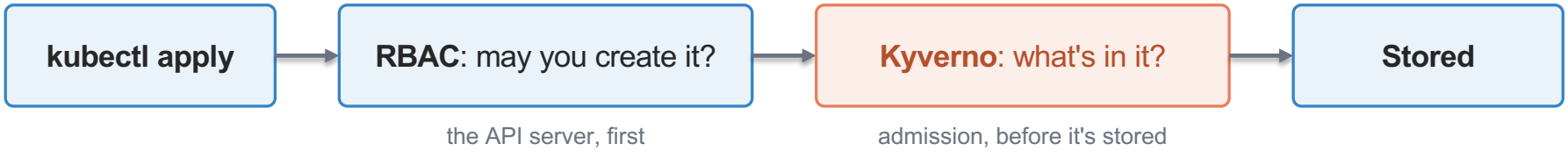


kind · NGINX Gateway Fabric 2.8 · Inference Extension v1.6 · llm-d router · Kyverno 1.19

Kyverno in 30 seconds

Policies are Kubernetes objects, written in CEL, one directory each

- **Kyverno**: A CNCF policy engine for Kubernetes.
- **CEL** (Common Expression Language): The language Kubernetes writes its own admission rules in. Kyverno adds lookups, like a team's list of models in a ConfigMap (`resource.Get`).
- **ValidatingPolicy**: Allows or denies a change at admission. Every policy in this session is one.




```
# policies/p2-restrict-model-access, abridged
kind: ValidatingPolicy
metadata: {name: restrict-model-access}
spec:
  matchConstraints: ...   # HTTPRoutes
  variables: ...          # the team's list
  validations:
  - expression: variables.denied.size() == 0
```

NGINX Gateway Fabric in 30 seconds

NGINX Gateway Fabric routes to InferencePools and checks that each route is valid

What it is

- An open source Gateway API implementation, with NGINX as the data plane.
- Its control plane watches Gateways and routes, and configures an NGINX Deployment for each Gateway.
- With the Inference Extension, it asks the endpoint picker which pod gets each request.

What it checks on its own

- A route to another namespace needs a ReferenceGrant: `RefNotPermitted` otherwise.
- It honours the pool's `failureMode`: FailClose (the default) or FailOpen.
- Problems show in the route's status, after it's stored.

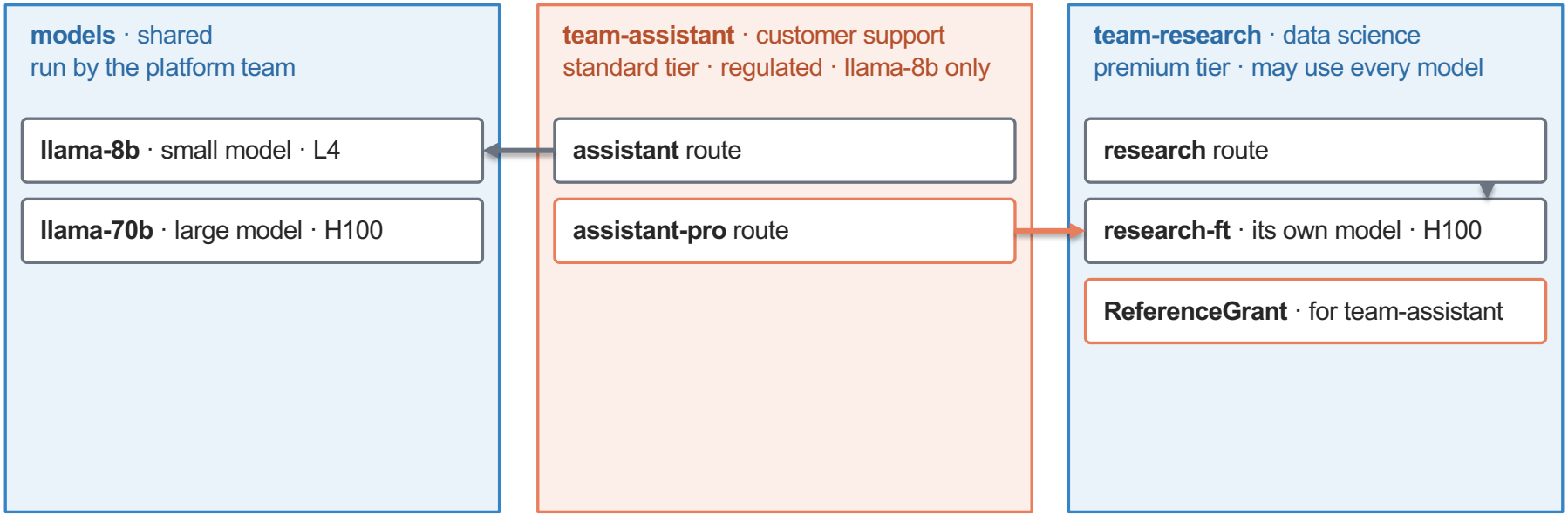
NGINX
Gateway Fabric

Every Gateway API implementation must check ReferenceGrants. The patterns today work with any of them.

Scenario 1 · Tenant isolation

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER

A friend shares their model, and RBAC can't tell it from a Service



Demo: Before the Kyverno policy ►

Scenario 1 · Tenant isolation

Three checks see that grant, and only Kyverno reads what it shares

RBAC · API server

May team-research create a ReferenceGrant?

```
$ kubectl auth can-i create \
    referencegrants -n team-
research
yes
```

Sees the kind and the verb, not what the grant points at.

Gateway · controller

Is the reference granted?

```
ResolvedRefs  True
# the grant exists, so
# the route resolves
```

Checks references, and reports in status after the object is stored.

Kyverno · admission

Does this grant share an InferencePool?

```
Denied by vpol-restrict-
inferencepool-referencegrants:
only the platform team ...
```

Reads the fields, and refuses before the object is stored.

Scenario 1 · Tenant isolation

Only the platform team may share InferencePools

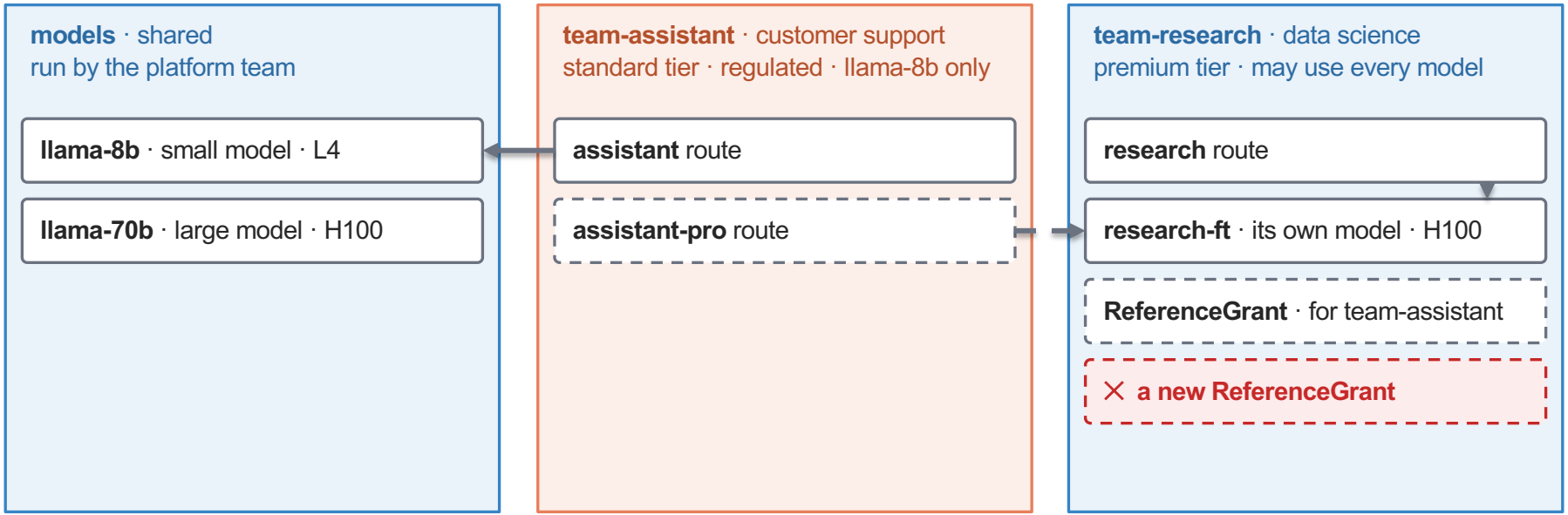
```
# policies/p1-restrict-inferencepool-referencegrants
kind: ValidatingPolicy
spec:
  autogen:
    validatingAdmissionPolicy: {enabled: true}
  matchConditions:
  - expression: >-
      object.spec.to.exists(t,
        t.kind == 'InferencePool')
  validations:
  - expression: >-
      'platform-admins' in request.userInfo.groups
```

- **A ReferenceGrant lets another namespace in.** A tenant role that can share a Service can share a model too.
- **Kyverno generates the ValidatingAdmissionPolicy (VAP).** One policy, enforced in the API server too.
- Plain CEL is enough here. It works even if Kyverno is unavailable.

A native VAP where plain CEL is enough; Kyverno for lookups, reports and cleanup.

Scenario 1 · Tenant isolation

Now the grant is refused before it's stored



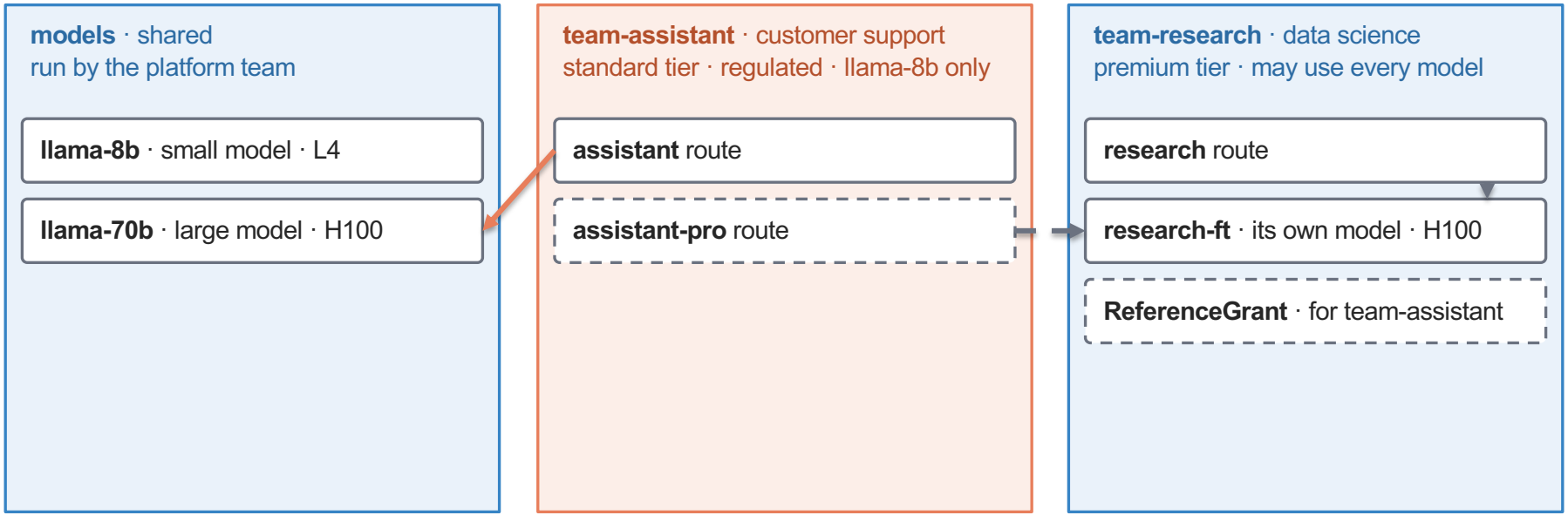
Got in before the policy and still runs: admission only checks new changes

Denied by vpol-restrict-inferencepool-referencegrants (native VAP): only the platform team can share InferencePools

Demo: With the Kyverno policy ►

Scenario 2 · Inference routing

team-assistant routes to the large model directly



team-assistant may edit its own route. The platform approved llama-8b only, and nothing checks the route against that.

Got in before the policy and still runs: admission only checks new changes

Demo: Before the Kyverno policy ►

Scenario 2 · Inference routing

Kyverno checks each route against the team's list of models

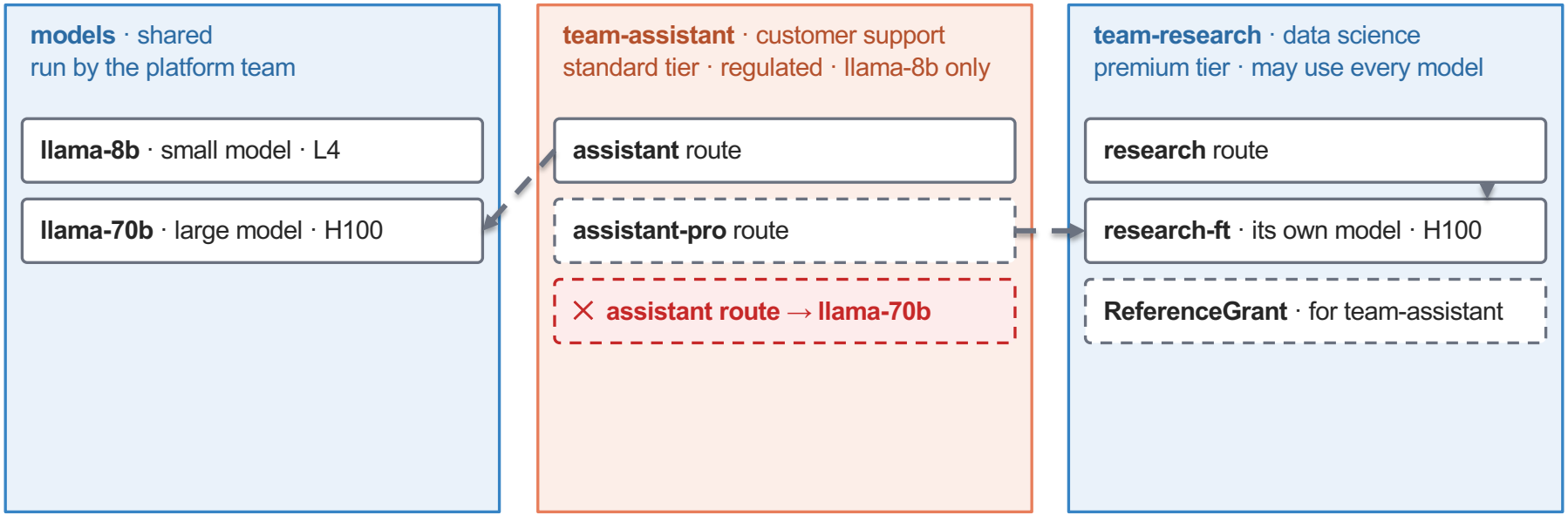
```
# restrict-model-access, abridged
# allowed: ConfigMap kyverno/model-allowlist
#   team-assistant: models/llama-8b
# pools: every InferencePool the route uses,
#   mirrors included
 pools.filter(p, !(p in allowed))
   .size() == 0
```

- **RBAC:** may team-assistant create an HTTPRoute? Yes.
- **The gateway:** is the reference granted? Yes: the platform's grant covers its shared models. Approvals are a separate list.
- **Kyverno:** is llama-70b on team-assistant's list? No.

The list is a ConfigMap only the platform can edit, keyed by namespace and pool name.

Scenario 2 · Inference routing

Now a route to an unapproved model is refused



Got in before the policy and still runs: admission only checks new changes

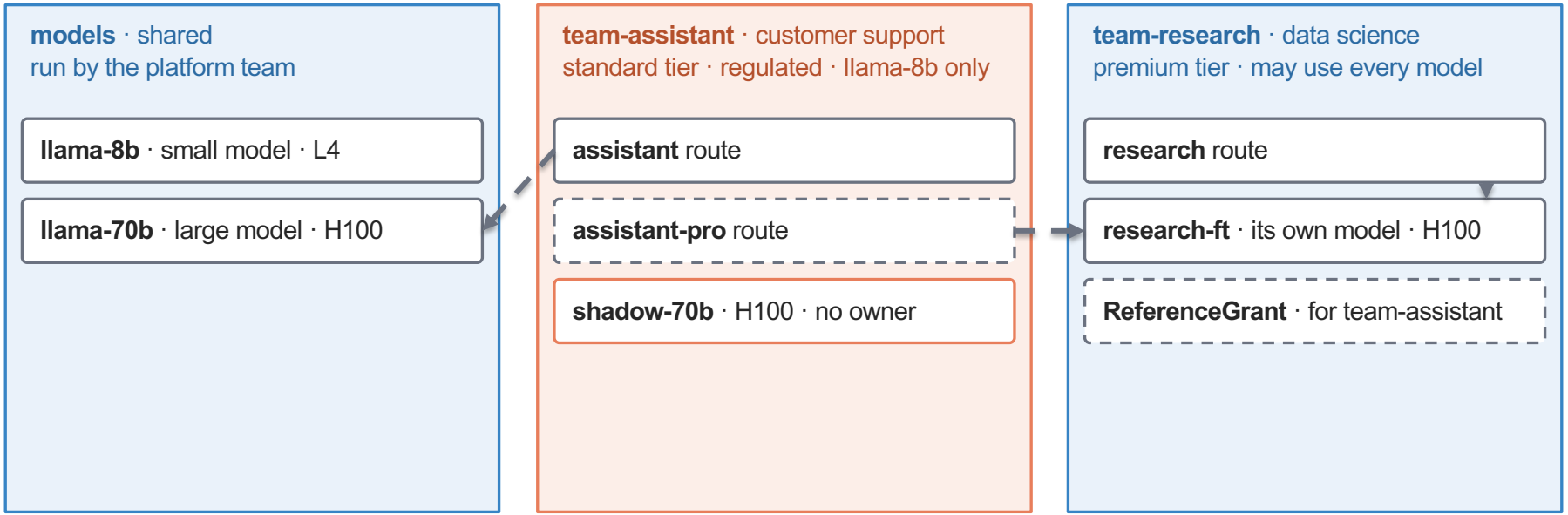
Denied by restrict-model-access: team-assistant is not approved for models/llama-70b: ask the platform team to add it to your list

Demo: With the Kyverno policy ►

Scenario 3 · Cost attribution

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER

team-assistant runs its own large model on an H100 nobody pays for



Got in before the policy and still runs: admission only checks new changes

team-assistant may create Deployments. RBAC doesn't look at the GPU class or the cost-center label.

Demo: Before the Kyverno policies ►

Scenario 3 · Cost attribution

Every GPU names who pays, and only premium teams get H100s

Who pays

```
# require-cost-center, abridged
# InferencePools, Pods and every workload kind
 object.metadata.?labels[?'cost-center']
   .orValue('') != ''
# ...and the same on the pod template
```

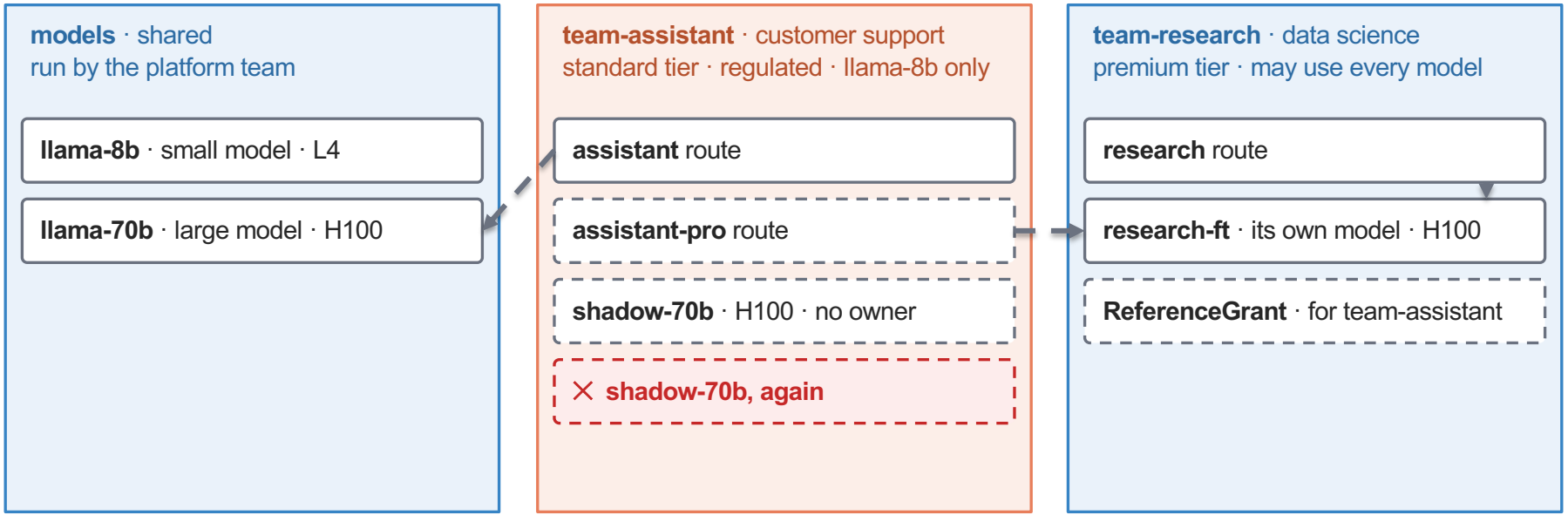
Which GPU class

```
# restrict-gpu-class-by-tier, abridged
# match: gpu.example.com/class: h100
#   (pod label or nodeSelector)
 variables.tier == 'premium'
# tier: the Namespace's label
```

- **RBAC and the gateway:** nothing to refuse. The Deployment and the pool are valid config.
- **Kyverno:** whose budget, and which GPU class for this tier. Pods, Deployments, StatefulSets and Jobs alike.

Scenario 3 · Cost attribution

Now the same model is refused twice: no owner, then the wrong tier



Denied by require-cost-center: Deployment shadow-70b needs a cost-center label

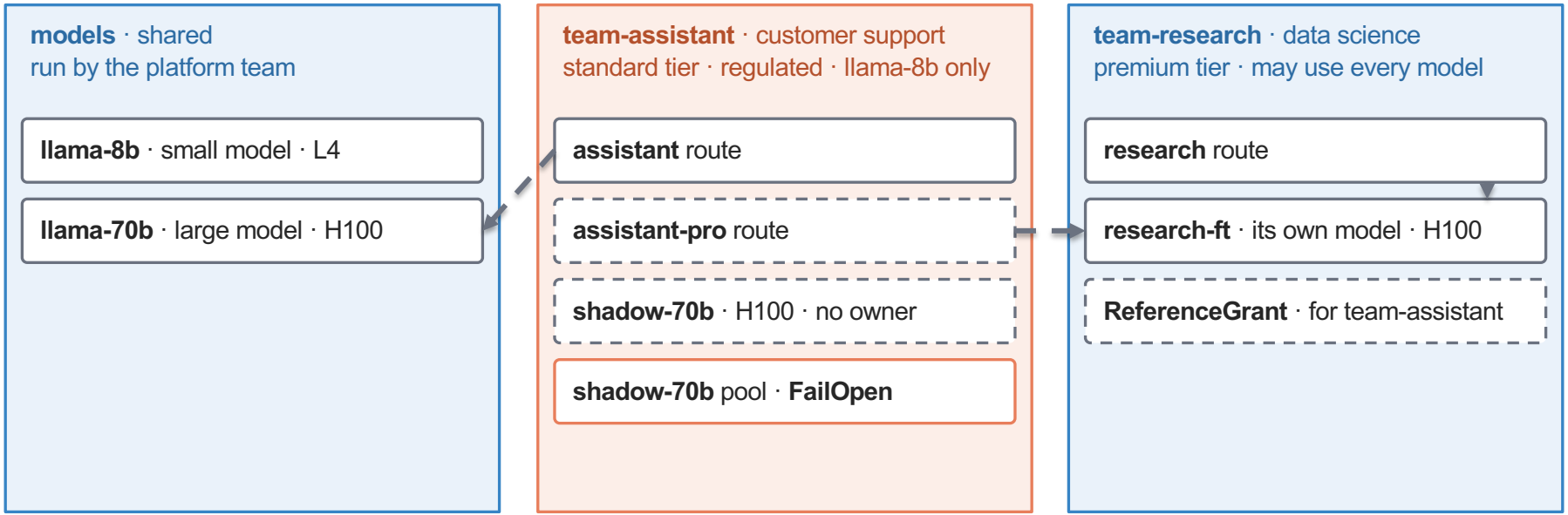
With the label: denied by restrict-gpu-class-by-tier: team-assistant (standard tier) can't use h100 GPUs

Got in before the policy and still runs: admission only checks new changes

Demo: With the Kyverno policies ►

Scenario 4 · Failover

team-assistant sets its pool to keep serving if the picker is down



team-assistant may edit its own InferencePool. RBAC doesn't look at failureMode, or at the namespace being regulated.

Got in before the policy and still runs: admission only checks new changes

Demo: Before the Kyverno policy ►

Scenario 4 · Failover

Regulated teams fail closed, whoever owns the pool

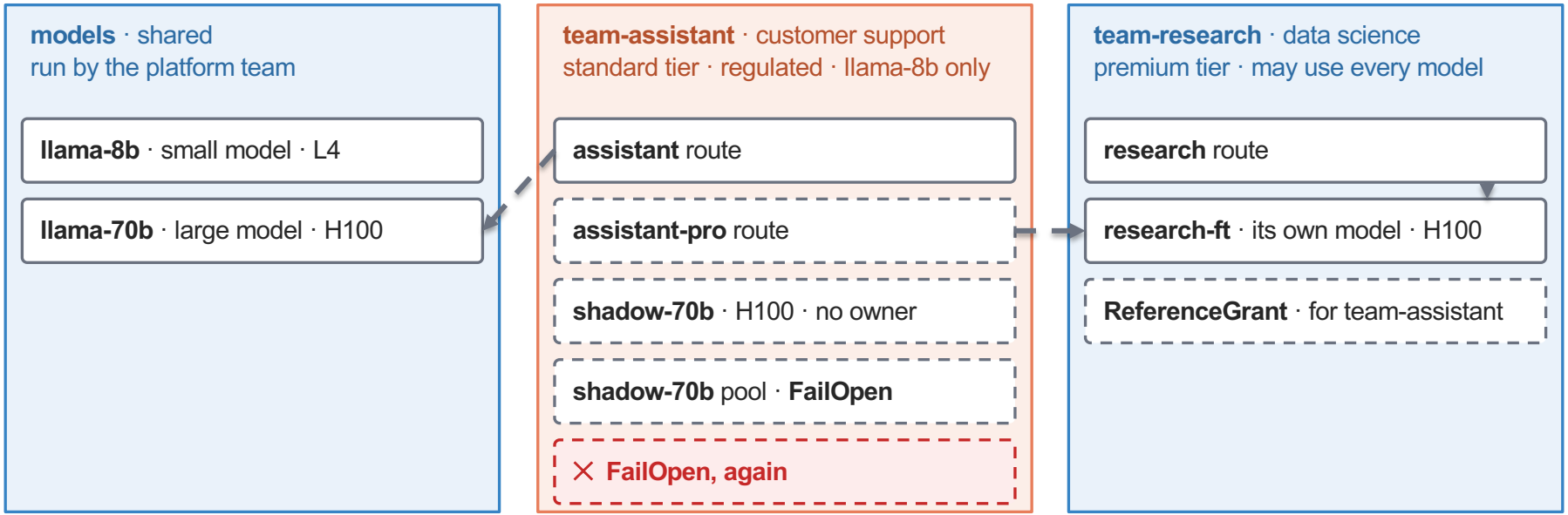
```
# deny-failopen-for-regulated, abridged
# match: failureMode: FailOpen
 !variables.regulated
# regulated: the Namespace's label,
# which only the platform can set
```

- **RBAC:** may team-assistant edit its pool? Yes.
- **The gateway:** FailOpen is valid, and NGINX Gateway Fabric does what it says.
- **Kyverno:** is this namespace regulated? Then no FailOpen.

The failure mode is part of the platform's contract with a regulated team, not the team's own choice.

Scenario 4 · Failover

Now FailOpen is refused for a regulated team



Denied by deny-failopen-for-regulated: team-assistant is regulated: FailOpen is not allowed (shadow-70b): requests must go through the endpoint picker; ask the platform team

Got in before the policy and still runs: admission only checks new changes

Demo: With the Kyverno policy ►

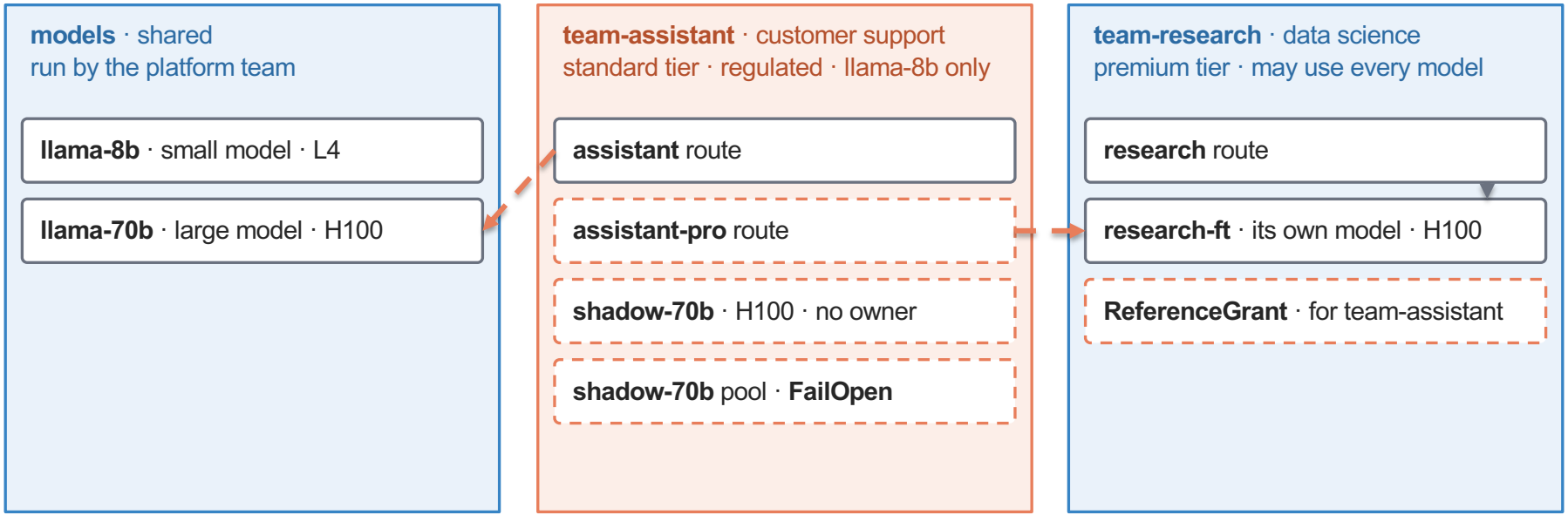
Scenario recap

RBAC works, but it’s not granular enough

Workaround	RBAC: may create…	Kyverno reads…
Borrow a model	a ReferenceGrant	`spec.to.kind`: is it an InferencePool?
Just point at the big one	an HTTPRoute	which pool, and is it on this team's list?
Shadow GPU	a Deployment	GPU class against the tier; `cost-center` on the pods
Keep serving	an edit to its own InferencePool	`failureMode`, and whether the namespace is regulated

But wait a minute!

Everything from before the policies is still running

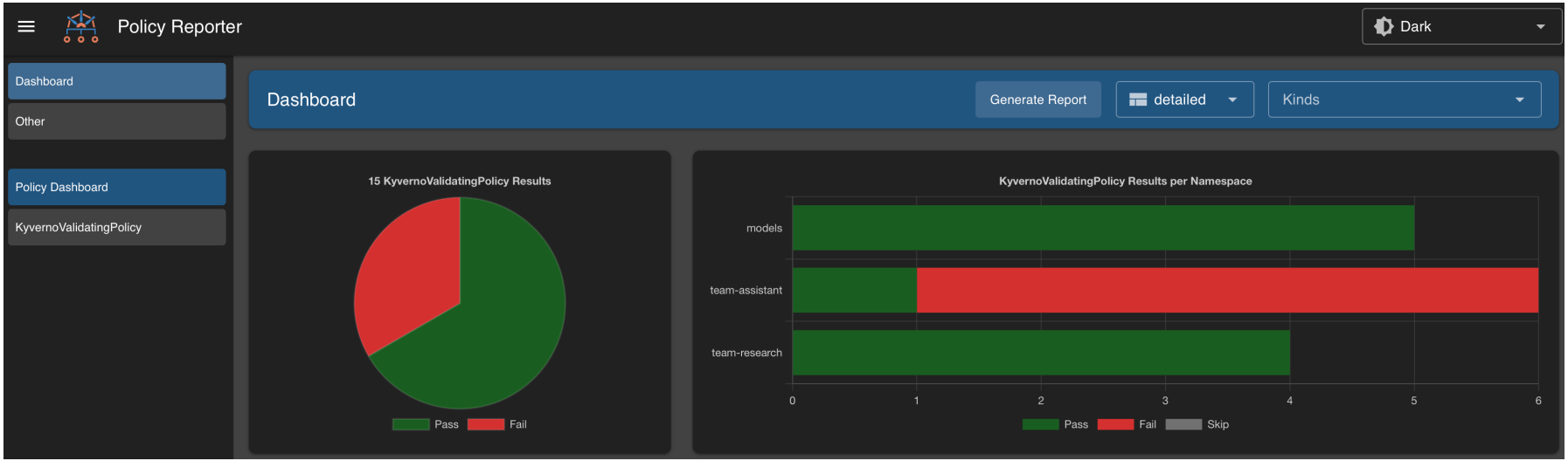


Got in before the policy and still runs: admission only checks new changes

Admission only checks new changes. Kyverno's background scans find and report each one.

Demo: Kyverno's background scans ►

Policy Reporter



Policy Reporter is Kyverno's dashboard. team-assistant has 5 failures: everything that got in before the policies. Filter by namespace or policy, and hand each team its list.

Clean up

A DeletingPolicy removes model servers nobody pays for

```
# policies/c1-cleanup-unowned-gpu-workloads
kind: DeletingPolicy
spec:
  schedule: "0 * * * *"
  matchConstraints:   # Deployments,
    ...               # StatefulSets, Jobs
  conditions:
  - expression: variables.isTenant
  - expression: <model server>
   - expression: <no cost-center label>
```

- A report finds the GPU nobody pays for. A **DeletingPolicy** removes it on a schedule and frees the GPU.
- Same test as the admission rule, so what's denied today is cleaned up tomorrow.
- Deleting is final: report first, tell the owners, then enable it.

In the repo, with tests. Not run on stage: delete a team's model only after its owners have seen the report.

Next steps

Find what RBAC can't see, audit, enforce, then clean up

1 List the gaps	For each object tenants may create: what in it does RBAC not see? Which pool, which GPU, which failure mode.
2 Write and test	One CEL rule each, with Kyverno CLI tests, like every policy in the repo.
3 Audit	Install with `validationActions: [Audit]`. Nothing breaks, and you see what it would deny.
4 Enforce	Switch to Deny, starting with the ReferenceGrant guard: it runs as a native ValidatingAdmissionPolicy.
5 Clean up	Tell the owners what's left in the report, then let a DeletingPolicy remove it.

Take these home

Everything from today is in the repo

github.com/nginx/nginx-demos

- The five policies from today (Kyverno CEL), each with CLI tests
- Every scenario as a working demo, plus more: cleanup with a DeletingPolicy, priority by tier, outside-API fallbacks, NetworkPolicies, rate limits

Pattern · policy in the repo
Guard who shares models `restrict-inferencepool-referencegrants`
Model allowlist by pool `restrict-model-access`
Every GPU names an owner `require-cost-center`
GPU class by tier `restrict-gpu-class-by-tier`
Regulated tenants fail closed `deny-failopen-for-regulated`

Book signing · F5 / NGINX (Booth 100) @ 4:15pm!

Kyverno Con 2026
NORTH AMERICA
CLOUD NATIVE THEATER



Understanding Kyverno in a visual way

Aurélie Vache

- Kyverno 1.19 in illustrated sketchnotes
- All five CEL policy types, including the ValidatingPolicy and DeletingPolicy from today!
- Policy reports, exceptions, testing and troubleshooting

Thank you

Questions?

github.com/nginx/nginx-demos

Ciara Stacke · Alessandro Fael Garcia
F5 / NGINX